# Euler's Gallery

WyomingWill's rigid-body curiosities in Bullet Physics Playground · 21 October 2026

An 18th-century walnut gallery, L-shaped, for WyomingWill's rigid-body curiosities. Walnut panelling with gilded frames, parquet floor, a coffered ceiling with chandeliers, the title in gilt on the wall at the corner, an armillary sphere in the corner and a tall window at the far end. Each exhibit stands in a bay of its own with a gilt-lettered placard.

**Only the exhibit you're standing at runs.** Walk away and it stops where it is (a gömböc frozen in mid-air, a top mid-flip); come back and it carries on from there.

**Every exhibit holds still for half a second whenever it starts**: when you first arrive at it, when you press R, and each time it starts itself again. It puts everything in place, then waits, so you see how it begins before anything moves. The physics stops too (a zero time step), so the pause changes nothing: an exhibit's console output is the same, line for line, with or without it. (The falling Slinky has its own hold: the hand lets go after half a second, or as long as its hold slider says.) `HOLD_S` sets the pause, in seconds; 0 turns it off.

Nothing appears out of thin air, either. Each exhibit is set up the moment the hall loads, so its bay is never empty when you walk up. And the gömböcs and the Billes, which are dropped, start resting on the table: they rest there through the pause; then an attendant's lift takes hold of each (0.2 s), carries it up to where it's let go (0.8 s), and lets go. (The lift waits for the pause to end, so that, as everywhere, the pause changes nothing.) When one is dropped again, it's carried up from where it came to rest. Each one's clock starts when it's let go.

## Running it

Open `gallery.lua` in bpp. The exhibits are in `exhibits/`, one folder each (`exhibits/Gomboc/`, `exhibits/Bille/` and so on); each also runs on its own from its folder. From the repository's top folder: `release/bpp -f demo/WyomingWill/EulersGallery/gallery.lua`.

## Keys

Click the 3D view first so it has the keyboard.

| Key | Action |
|---|---|
| `Tab` | walk on: the view from the entrance, the exhibits in wing A, the view from the corner down wing B, the exhibits in wing B, then back to the entrance |
| `Shift+Tab` | walk back |

On Tab and Shift+Tab the camera flies to the next stop, out of the bay, along the aisle (round by the corner between the wings) and into the next bay, in 1.2 to 3.5 seconds. The exhibit you're going to starts at once; its own camera moves wait until you've arrived. `GALLERY_FLY=0` jumps instead, as before. The way stays inside the gallery's walls (`GALLERY_FLY_LOG=1` prints each flight's extent, or `=file` writes it there).

Everything else goes to the exhibit you're at, with the same keys as when it runs on its own; the Shortcuts pane shows them. The Params pane shows only its sliders, named after it ("Variety: speed", "Tippe top: friction"); the others keep their settings for when you come back.

Rest the mouse on an exhibit, a placard, a covered bay or the armillary sphere to see what it is.

To look closer: the wheel moves you a tenth of the way in towards what the bay is showing at each click (and back out the same way); drag with the left button to walk round it, and with the right to slide the view. To turn round something else, a dial say, Shift + right click on it first (on empty space, it's the middle of the hall again; walking on

puts it back on each bay as you arrive). (This needs bpp from 14 October 2026 or later; before, the mouse turned round the middle of the whole hall.)

Looking down the hall, the Params pane has one slider, "Gallery: stop": where you are (1 is the entrance, then on round as Tab goes). Drag it to go straight to an exhibit. bpp's Restart (Ctrl+R) runs the whole hall again from the start, every exhibit fresh, and then puts every slider back as it was, this one included, so you're back where you were.

bpp's own one-letter keys (`S` stop, `D` sleeping, `R` reload, `P` POV-Ray) would change the whole hall, so at an exhibit they do only what the exhibit uses them for. Looking down the hall, `P` is held back (use the POV-Ray menu).

bpp's markers on joints (`v.showConstraints`) are off looking down the hall; at an exhibit they're as the exhibit has them (the gyroscopes turn them off).

## How faithful it is

`report/physics-fidelity-report.pdf` goes through every exhibit: how it is simulated, and each check of it against theory, with the numbers. The sources it is built from, and how to rebuild it, are in `report/`.

## The plan

Wing A runs north from the entrance to the corner; wing B runs east from the corner. Each wing has bays on both sides, 4.2 m wide and 3.3 m deep, with a 3.4 m aisle between.

Each wing has an idea, on a gilt board hung over its aisle where it begins:

- **Wing A, "Turning": things that roll, tumble and spin**, where the body's shape and spin decide what it does. The bays facing each other across the aisle make pairs:
- shapes that right themselves (Gömböc, Bille);
- rolling down, and rolling "up" (Galileo's ramp, the double cone);
- rolling that isn't simple (the oloid, Euler's disk);
- spin that changes on its own (the rattlebacks, the spinning eggs);
- things that flip over (the tippe tops, the Dzhanibekov effect);
- precession (the gyroscopes, the bicycle wheel).
- **Wing B, "Swinging": pendulums, collisions and many bodies**, ending with gravity at the largest scale. In order:
- Newton's cradle and Huygens' clocks;
- Kapitza's pendulum and the double pendulum;
- pendulums with a twist and the chain fountain;
- the falling Slinky and the Brazil-nut effect;
- the solar system, as the finale.

| Bay | Exhibit | Folder |
|---|---|---|
| A, west 1 | Gömböc Variety | `exhibits/Gomboc/gomboc-variety.lua` |
| A, east 1 | Bille and the polyhedra | `exhibits/Bille/bille-drop.lua` |
| A, west 2 | Galileo's ramp | `exhibits/GalileoRamp/galileo-ramp.lua` |
| A, east 2 | the double cone | `exhibits/DoubleCone/double-cone.lua` |
| A, west 3 | the oloid and the sphericon | `exhibits/Oloid/oloid.lua` |
| A, east 3 | Euler's disk | `exhibits/EulersDisk/eulers-disk.lua` |
| A, west 4 | the rattlebacks | `exhibits/Rattleback/rattleback.lua` |
| A, east 4 | the spinning eggs | `exhibits/SpinningEgg/spinning-egg.lua` |
| A, west 5 | the tippe tops | `exhibits/TippeTop/tippe-top.lua` |

| Bay | Exhibit | Folder |
|---|---|---|
| A, east 5 | the Dzhanibekov effect | `exhibits/Dzhanibekov/dzhanibekov.lua` |
| A, west 6 | the gyroscopes | `exhibits/Gyroscope/gyroscope.lua` |
| A, east 6 | the bicycle wheel | `exhibits/BicycleWheel/bicycle-wheel.lua` |
| B, north 1 | Newton's cradle | `exhibits/NewtonsCradle/newtons-cradle.lua` |
| B, south 1 | Huygens' clocks | `exhibits/HuygensClocks/huygens-clocks.lua` |
| B, north 2 | Kapitza's pendulum | `exhibits/Kapitza/kapitza.lua` |
| B, south 2 | the double pendulum | `exhibits/DoublePendulum/double-pendulum.lua` |
| B, north 3 | pendulums with a twist | `exhibits/PendulumsTwist/pendulums-twist.lua` |
| B, south 3 | the chain fountain | `exhibits/ChainFountain/chain-fountain.lua` |
| B, north 4 | the falling Slinky | `exhibits/Slinky/falling-slinky.lua` |
| B, south 4 | the Brazil-nut effect | `exhibits/BrazilNut/brazil-nut.lua` |
| B, north 5 | the solar system | `exhibits/SolarSystem/solar-system.lua` |

Gömböc Drop C (`exhibits/Gomboc/gomboc-drop-c.lua`) isn't in the hall: Gömböc Variety shows Gomboc-C and more.

## Adding exhibits

Each exhibit is one line in `EXHIBITS` near the top of `gallery.lua`:

```
lua { dir = "exhibits/TippeTop/", file = "tippe-top.lua", name = "the tippe tops", label = "tippe-top", prefix = "Tippe top: ", floorY = -75, about = "The tippe top: ..." },
```

| Setting | What it does |
|---|---|
| `dir`, `file` | the script (`dir` from this folder). If it isn't there yet, the bay is covered with a dust sheet and the placard says "in preparation"; when the script turns up, the bay comes to life. |
| `name` | what the header and Tab call it |
| `label` | its placard: a key of `gallery-meshes/labels.lua` (the lettering is made by `make-labels.py`) |
| `prefix` | its sliders in the Params pane are named `prefix .. name` |
| `floorY` | the height of its own ground plane, in its own units: the hall's floor goes there |
| `stand` | `{ w, d }`: a walnut stand under it, for exhibits (the trays) that float above their ground plane on their own |
| `about` | what its placard says when the mouse rests on it |

Add lines and the hall grows to fit: both wings get longer. `SPARE_BAYS` adds empty bays at the far end (0 now: twenty-one exhibits, twelve bays in wing A and nine in B, the last row's other bay empty).

An exhibit is written as a script that runs on its own, the way the existing ones are: centimetres, its viewing side toward +z, a `Plane` for its ground, its camera set once as it loads, and its physics stepping done in `v:preSim` (the gallery passes its physics settings on while you're at it).

## How it works

Each exhibit is the unchanged script, loaded into a sandbox of its own (its own set of globals) that:

- puts everything it builds into a Bullet world of its own, untouched, at its own coordinates (bpp draws that world in its bay, turned to face the aisle, and moves its camera there);
- finds its files in its own folder;

- gives it its own random numbers;
- keeps its sliders apart, and passes its keys, callbacks, help text and hover only while you're at it (hover works on any exhibit);
- keeps its ground plane, in its own world, not drawn (the hall's floor is drawn there, at the same height).

**Each exhibit runs exactly as it does on its own.** Each one has a Bullet world of its own (bpp's `v:newWorld()`), made exactly as bpp makes the scene's world, holding only what that exhibit builds, at its own coordinates; bpp draws it in its bay (`v:placeWorld()`). It also gets the same random numbers it would on its own. So it gives the same numbers, digit for digit -- checked for every one of them by running each both ways and comparing everything it prints (with a fixed seed for the ones that seed from the clock: `GC_SEED`, `GV_SEED`, `BD_SEED`). Walking away for a while and coming back gives the same numbers as never leaving.

Why separate worlds: Bullet works out an exhibit's contacts in an order that depends on everything else in its world, even things that never touch it, and it rounds differently 15 m from the origin than at it. Either is enough to send a rattleback, a tippe top or a chain a different way within seconds. In a world of its own, at its own coordinates, nothing else is there.

**Needs bpp with the separate-worlds patch** (`v:newWorld`, `v:useWorld`, `v:placeWorld`, `v:currentWorld`; and `v:showParams`, which keeps the Params pane to the exhibit you're at -- without it, every exhibit's sliders are listed). A script that doesn't use them -- every other demo, the rec room, each exhibit on its own -- runs exactly as before. (POV-Ray export writes each exhibit where its world has it, at the origin, not in its bay.)

Only the world of the exhibit you're at is stepped; the others wait exactly as they are, and cost nothing. Each exhibit's own step sizes apply while you're at it; its other physics settings belong to its world.

`GALLERY_START=<part of a name>` starts at that exhibit (`corner` for the view down wing B), for trying one out. (`CABINET_START`, its old name, still works.) The hall's console lines start with `GALLERY:`.

**POV-Ray.** The gallery renders in POV-Ray (File, or `bpp -e`) as on the screen. This needs a bpp that exports each object moved to where its world is drawn (an exhibit's world is placed in its bay with `v:placeWorld`); an older bpp exports every exhibit at its own coordinates, piled up in the middle of the hall. The exhibits' ground planes, invisible on the screen, are left out of the scene: in POV-Ray a plane has no edges and would cover the floor.

## Files

- `gallery.lua`: the hall. Open this one.
- `gallery-meshes/`: the gilt lettering (title and placards), `labels.lua` (their sizes), and the brass ring of the armillary sphere.
- `make-labels.py`: makes the lettering (needs Python with Pillow and NumPy and the DejaVu fonts).
- `exhibits/`: the 21 exhibits, a folder each, with its README.
- `report/`: the physics fidelity report and its sources.